

MEDASSIST AI

AI-BASED HEALTHCARE ASSISTANCE AND DISEASE PREDICTION PLATFORM

FINAL PROJECT DOCUMENTATION

Infosys Springboard AI Internship Project

Developed by

Swastik Sukla

B.Tech – Computer Science and Engineering

ITER, Bhubaneswar

Expected Graduation: 2027

Project Duration: 8 Weeks

Final Submission – Milestones 1 to 4

DECLARATION, ACKNOWLEDGEMENT & PROJECT OVERVIEW

Declaration

I hereby declare that the project titled “MedAssist AI – AI-Based Healthcare Assistance and Disease Prediction Platform” was developed as part of my academic and internship learning work. The system integrates a web-based healthcare interface with a FastAPI backend, database services, and a machine learning disease prediction component. The project was developed, tested, documented, and deployed during the project period.

Acknowledgement

I sincerely thank Infosys Springboard and the mentors associated with the internship for providing the opportunity to work on an applied artificial intelligence project. I also thank my institution, faculty members, and everyone who supported the learning, development, testing, and deployment activities.

Project Overview

MedAssist AI is a web-based healthcare assistance platform designed to analyze entered symptoms and provide a potential disease prediction with a confidence value, risk assessment, and healthcare-oriented recommendations. The platform also provides patient and doctor dashboards, prediction history, healthcare analytics, health trends, and prediction reports.

The project was completed progressively over four milestones. The first three milestones established the application, healthcare workflows, machine learning prediction, recommendation features, analytics, and reporting. Milestone 4 focused on containerization, cloud deployment, production integration, verification, documentation, and final demonstration.

Keywords

Artificial Intelligence, Machine Learning, Healthcare, Disease Prediction, FastAPI, React.js, Scikit-learn, Docker, AWS EC2, Vercel, SQLite, Healthcare Analytics.

ABSTRACT

MedAssist AI is an AI-enabled healthcare assistance platform developed to provide a structured digital workflow for symptom analysis, potential disease prediction, risk assessment, and healthcare-oriented advisory information. The platform combines a React.js frontend with a Python FastAPI backend and a machine learning model built for symptom-based disease prediction. Patient and doctor workflows are separated through role-based application interfaces.

The patient workflow allows a user to register, log in, maintain profile and medical-history information, enter symptoms, obtain a model-generated prediction, and review prediction history and reports. The doctor workflow provides access to registered patients, patient records, prediction history, risk information, recommendations, treatment/advisory content, analytics, and health trends.

Milestones 1–3 established the application, prediction workflow, recommendations, risk assessment, analytics, trends, and reporting. Milestone 4 containerized the backend using Docker, deployed it on AWS EC2, deployed the React frontend through Vercel with GitHub-based continuous deployment, implemented proxy routing to address browser mixed-content restrictions, and verified end-to-end cloud communication.

Main Outcomes

- Integrated patient and healthcare-provider workflows.
- Machine-learning based symptom-to-disease prediction with confidence scoring.
- Risk assessment and disease-specific healthcare recommendations.
- Analytics, prediction history, health trends, and prediction reports.
- Dockerized FastAPI backend with bundled ML model files.
- AWS EC2 backend and Vercel frontend deployment.
- Cloud verification of registration, authentication, symptom submission, and real-time ML prediction.

1. INTRODUCTION, PROBLEM STATEMENT & OBJECTIVES

1.1 Introduction

Healthcare applications increasingly use digital systems to organize patient information and support clinical workflows. Machine learning can assist by identifying patterns in symptom data and generating potential disease predictions. However, a useful application requires more than a prediction model: it needs data management, user interfaces, backend APIs, clear result presentation, and deployment infrastructure.

1.2 Problem Statement

Patients may find it difficult to interpret a collection of symptoms or organize previous prediction information in one place. A basic machine learning script can produce a prediction, but it does not by itself provide authentication, patient records, dashboards, recommendations, analytics, or a deployable web platform. The project therefore aims to build an integrated platform connecting symptom analysis with a complete digital workflow.

1.3 Objectives

1. Develop a web-based healthcare assistance platform.
2. Provide patient registration and authentication workflows.
3. Allow users to maintain profile and medical-history information.
4. Accept symptoms and use a trained ML model for potential disease prediction.
5. Display prediction confidence and structured risk assessment.
6. Provide disease-specific recommendations, treatment/advisory information, and precautions.
7. Maintain prediction history and generate prediction reports.
8. Provide healthcare providers with patient records and analytics.
9. Visualize health trends and prediction information.
10. Deploy frontend and backend to cloud infrastructure for public demonstration.

1.4 Scope

The project covers symptom-based disease prediction, patient information management, healthcare-provider viewing workflows, analytics, reporting, and cloud deployment. The platform is an educational and decision-support prototype and does not replace professional medical diagnosis.

2. PROPOSED SYSTEM & DEVELOPMENT MILESTONES

2.1 Proposed System

The proposed platform connects the user interface, backend APIs, database, and machine learning model into a single workflow. A patient enters profile, medical history, and symptoms through React. The backend validates and stores relevant information and invokes the trained prediction model. The result is processed into risk, recommendation, treatment/advisory, and reporting information.

2.2 Major System Modules

Module	Purpose
Authentication	Registration, login, role-based workflows.
Patient Profile	Stores and displays profile information.
Medical History	Maintains history entered by the patient.
Symptom Analysis	Accepts symptoms for prediction.
Disease Prediction	Generates potential disease and confidence.
Risk Assessment	Maps prediction confidence into risk categories.
Recommendation Engine	Provides disease-specific guidance.
Prediction History	Stores and displays previous predictions.
Doctor Dashboard	Patient records and healthcare-provider workflow.
Analytics	Aggregate patient and prediction statistics.
Health Trends	Visualizes prediction/trend information.
Prediction Reports	Presents structured prediction information.

2.3 Milestone Progression

Milestone	Major Deliverables
1	Core application, authentication, database, patient and doctor dashboards.
2	Symptom processing, ML prediction, patient records and prediction history.
3	Recommendations, risk assessment, treatment/advisory, analytics,

	trends and reports.
4	Docker, AWS EC2, Vercel, proxy routing, production verification, documentation and final demonstration.

3. SYSTEM ARCHITECTURE & WORKFLOW

3.1 High-Level Architecture

The final system follows a layered architecture. The React frontend provides the user-facing application. Requests are routed to the FastAPI backend. The backend manages authentication and application logic, communicates with the database, and loads the serialized machine learning model. In deployment, the backend runs in a Docker container on AWS EC2 while the frontend is delivered through Vercel.

3.2 Logical Architecture

Layer	Components	Responsibility
Presentation	React.js, Vite, CSS	Dashboards, forms, navigation, result and analytics display.
Application	FastAPI, Python	API endpoints, validation, business logic and prediction.
Data	SQLAlchemy, SQLite	Persistent application data.
AI	Scikit-learn, Joblib, NumPy	Model loading and prediction.
Deployment	Docker, AWS EC2, Vercel	Containerization, cloud hosting and frontend delivery.

3.3 End-to-End Workflow

11. Open deployed React application.
12. Register or log in.
13. Enter profile and medical history.
14. Submit symptoms.
15. Frontend sends request through /api routing.
16. FastAPI processes the request.
17. ML model generates potential disease prediction.
18. Backend returns prediction and confidence.
19. Risk, recommendation and advisory information are generated.
20. Prediction is stored for history and review.
21. Doctor can review patient records and predictions.
22. Analytics and health trends aggregate stored information.

3.4 Deployment Flow

The deployed frontend is delivered through HTTPS. Requests beginning with /api are routed through the Vercel edge proxy to the AWS EC2 backend. This resolved the browser restriction that occurred when an HTTPS frontend attempted to call an HTTP API directly.

4. TECHNOLOGY STACK & DEVELOPMENT ENVIRONMENT

Category	Technology	Use
Programming	Python	Backend and ML integration.
Backend	FastAPI	REST API/application service.
Frontend	React.js + Vite	Web application and dashboards.
Database	SQLite	Persistence in deployed prototype.
ORM	SQLAlchemy	Database models/access.
ML	Scikit-learn	Disease prediction.
ML Utility	Joblib	Model serialization/loading.
Processing	NumPy, Pandas	Data/feature processing.
Containerization	Docker	Backend packaging.
Cloud	AWS EC2	Backend cloud server.
Frontend Hosting	Vercel	Frontend hosting and CI/CD.
Version Control	Git + GitHub	Source control and deployment integration.
API Testing	Swagger/OpenAPI	Interactive API documentation/testing.

4.2 Development Environment

- IDE: Visual Studio Code.
- Backend deployment runtime: Python 3.11 inside Docker.
- Frontend: React.js/Vite with Node.js and npm.
- Backend service: FastAPI/Uvicorn.
- GitHub main branch connected to Vercel.
- Cloud API service exposed through the EC2 host.

4.3 Core Backend Dependencies

The production backend container includes FastAPI, Uvicorn, Scikit-learn, Joblib, Pydantic, email-validator and related runtime dependencies.

4.4 Security Considerations

Role-based workflows, cloud security groups, authentication, input validation, CORS/proxy behavior, and separation of frontend and backend responsibilities were considered. Private credentials and secrets are not included in this report.

5. MILESTONE 1 – CORE PLATFORM DEVELOPMENT

5.1 Overview

Milestone 1 established the foundation of MedAssist AI through a functional web application, backend service, database persistence, authentication, and separate patient and doctor workflows.

5.2 Authentication and Roles

Registration and login were implemented as core entry points. The application stores user information and uses the selected role to provide an appropriate dashboard.

5.3 Patient Dashboard

- Patient login and dashboard access.
- Patient profile management.
- Medical-history information.
- Symptom entry interface.
- Prediction-related information.
- Prediction history and report areas.

5.4 Doctor Dashboard

- Healthcare-provider login.
- Patient list and patient selection.
- Patient records and prediction history.
- Dashboard summary information.
- Foundation for later analytics and review.

5.5 Database Foundation

SQLAlchemy models with SQLite persistence were used. The database structure included users, patients, symptoms, and predictions, allowing information to persist across requests.

5.6 Backend Foundation

FastAPI provided the Python API layer for registration, login, profile operations, symptom submission, prediction requests, and patient/record retrieval.

5.7 Outcome

Milestone 1 produced the working base platform on which machine learning and healthcare analytics functionality could be added.

6. MILESTONE 2 – AI PREDICTION & HEALTHCARE WORKFLOW

6.1 Machine Learning Integration

Milestone 2 integrated the trained disease prediction model into the FastAPI application. Serialized model artifacts were loaded by the backend so submitted symptom features could be processed without retraining the model during normal use.

6.2 Prediction Workflow

23. Patient enters symptoms.
24. Symptom information is converted into the feature representation expected by the model.
25. FastAPI loads and executes the trained model.
26. When probability information is available, the maximum probability is used as confidence.
27. Prediction and confidence are returned to the frontend.
28. Prediction is stored for later history and review.

6.3 Patient Records

Symptoms, medical history, and prediction history were connected with patient records, creating a continuous healthcare workflow.

6.4 Doctor Review

Healthcare providers can select a patient and view associated records and prediction information.

6.5 Confidence

Confidence is based on the model output and is not artificially changed. The application uses confidence as one input to its risk categorization.

6.6 Outcome

Milestone 2 established the central AI workflow: symptom input → model prediction → confidence → stored prediction → patient/doctor review.

7. MILESTONE 3 – RECOMMENDATION, RISK & ANALYTICS

7.1 Recommendation Engine

A healthcare recommendation engine was introduced to map predicted disease labels to disease-specific guidance. The purpose is to provide contextual information after prediction rather than only a disease label.

7.2 Risk Assessment

The application uses defined confidence thresholds to categorize results for dashboard presentation, including High Risk, Moderate Risk, Low Risk, and Needs Review.

7.3 Treatment and Advisory

Structured treatment/advisory information and precautions were associated with predictions. The interface includes caution that users should consult qualified healthcare professionals.

7.4 Analytics

Analytics	Purpose
Total Patients	Number of patients stored.
Total Predictions	Number of prediction records.
Risk Distribution	Breakdown by risk category.
Disease Distribution	Distribution of predicted disease labels.
Average Confidence	Aggregate prediction confidence.
Recent Predictions	Recent prediction activity.

7.5 Health Trends

Health trend visualization presents prediction history and confidence-related information over time.

7.6 Prediction Reports

Structured reports contain patient information, clinical context, prediction, confidence/risk, recommendation, treatment/advisory information, precautions, and a disclaimer.

7.7 Outcome

Milestone 3 transformed the prediction prototype into a more complete healthcare assistance platform with contextual guidance, analytics, trends, and reporting.

8. MACHINE LEARNING METHODOLOGY

8.1 Model Integration

The deployed application uses serialized machine learning artifacts. The model file, feature information, and class information are bundled with the backend deployment, making the prediction service self-contained.

8.2 Model Files

File	Purpose
disease_model.pkl	Serialized trained disease prediction model.
disease_features.pkl	Feature information required for model input.
disease_classes.pkl	Class-label information for model output interpretation.

8.3 Prediction Processing

The backend prepares the incoming symptom representation according to the feature structure expected by the trained model. The model generates a disease class. When probability output is available, the highest probability is used for the displayed confidence percentage.

8.4 Disease Mapping

The ML model produces a disease class while the application layer maps that class to risk and healthcare guidance. This separates prediction from how results are communicated.

8.5 Model Testing and Validation

Milestone 4 requires reporting accuracy, precision, recall, F1-score and a confusion matrix using appropriate evaluation data. Exact numerical metrics should be inserted from the actual evaluation run and must not be fabricated.

8.6 Healthcare Interpretation

A machine learning prediction is probabilistic and should not be treated as a confirmed diagnosis. MedAssist AI presents results as potential conditions and provides advisory information.

9. DATABASE & BACKEND/API ARCHITECTURE

9.1 Database Design

The application uses SQLAlchemy as the ORM and SQLite as the database for the deployed prototype. The database stores user, patient, symptom, and prediction information.

Entity	Representative Information
Users	Identity, login-related information and role.
Patients	Profile information associated with a user.
Symptoms	Symptoms submitted for analysis.
Predictions	Disease, confidence, risk and advisory information.

9.2 Backend Responsibilities

- Receive and validate requests.
- Authenticate users and process registration/login.
- Persist patient and prediction information.
- Load and execute the ML model.
- Generate recommendation/advisory information.
- Provide analytics and trends.
- Provide prediction reports.
- Serve the API through FastAPI/Uvicorn.

9.3 Representative API Endpoints

Endpoint	Function
/register	Register a user.
/login	Authenticate a user.
/patients	Retrieve patients.
/patient-records/{patient_id}	Retrieve patient records.
/predict-disease	Run disease prediction.
/prediction-history/{user_id}	Retrieve prediction history.
/analytics	Return healthcare analytics.
/health-trends/{user_id}	Return trend information.

/prediction-report/{prediction_id}	Return prediction report.
------------------------------------	---------------------------

9.4 API Documentation

FastAPI provides interactive OpenAPI/Swagger documentation. The deployed API documentation was used to inspect and test backend availability.

10. FRONTEND & DASHBOARD IMPLEMENTATION

10.1 React Application

The frontend was developed using React.js with Vite. The application is divided into role-specific workflows and reusable interface components, with CSS used for dashboards, cards, tables, navigation, reports, and responsive presentation.

10.2 Patient Interface

The Patient Dashboard provides profile, medical history, symptom submission, prediction results, prediction history, reports, and health-related visualizations.

10.3 Doctor Interface

The Doctor Dashboard provides patient lists, selected patient records, prediction information, risk assessment, recommendations, treatment/advisory sections, reports, analytics, and health trends.

10.4 Analytics

Analytics cards and visualizations summarize stored information. Risk and disease distributions provide aggregate views, while recent predictions and trends support review.

10.5 Reports

Prediction reports consolidate patient context, clinical information, model prediction, confidence/risk, recommendation, treatment/advisory information, precautions, and a disclaimer.

10.6 Production Integration

Hardcoded localhost API addresses were removed from client components and replaced with relative /api paths so the deployed frontend could communicate through the Vercel proxy.

11. MILESTONE 4 – BACKEND CONTAINERIZATION WITH DOCKER

11.1 Objective

The backend was packaged into a production-oriented Docker container containing the application, Python runtime, dependencies, database, and machine learning artifacts.

11.2 Container Contents

- FastAPI application code.
- Python 3.11 runtime.
- SQLite database used by the deployed prototype.
- `disease_model.pkl`, `disease_features.pkl` and `disease_classes.pkl`.
- FastAPI, Uvicorn, Scikit-learn, Joblib, Pydantic, email-validator and required dependencies.

11.3 Port Configuration

FastAPI listens on port 8000 inside the container. The deployment maps the service to the EC2 host for API access.

11.4 Automatic Recovery

The Docker container was launched using the `--restart always` policy, allowing Docker to restart the application after an unexpected stop or host restart.

11.5 Deployment Benefit

Containerization reduces differences between development and production environments and packages the ML runtime with the backend.

11.6 Resource Challenge

The free-tier EC2 environment experienced out-of-memory failures during build/runtime operations. A 2 GB virtual swap file was created and mounted at `/swapfile`, providing additional virtual memory and enabling the deployment process to complete more reliably.

12. MILESTONE 4 – AWS EC2 CLOUD INFRASTRUCTURE

12.1 EC2 Provisioning

The FastAPI backend was deployed to an AWS EC2 instance. The instance provided the public server environment required to host the Dockerized backend.

12.2 Network Configuration

Port	Purpose	Use
22	SSH	Remote server administration.
80	HTTP	Web/API HTTP access.
8000	API	FastAPI service access during deployment/testing.

12.3 Public Backend

The EC2 server was assigned a public IP address and used to verify that the FastAPI service and Swagger documentation were accessible from outside the local development environment.

12.4 Memory Optimization

A 2 GB swap file was created and mounted to address resource limitations and OOM crashes encountered during Docker build/runtime operations.

12.5 Docker Runtime

The Dockerized FastAPI service was run on EC2 with the model artifacts and backend dependencies available inside the container.

12.6 Operational Reliability

The restart policy and controlled security-group configuration provided basic operational reliability for the academic demonstration environment.

13. MILESTONE 4 – VERCEL, CI/CD & NETWORK ROUTING

13.1 Frontend Deployment

The React frontend was deployed using Vercel. The GitHub repository main branch was connected to the Vercel project, enabling automated continuous deployment.

13.2 Project Root

The Vercel project root was configured specifically as frontend/ so the correct React/Vite application was built.

13.3 Build Dependency Resolution

A build-time dependency problem related to Tailwind CSS/PostCSS was resolved by installing and locking @tailwindcss/postcss in devDependencies.

13.4 Mixed Content

The Vercel frontend uses HTTPS while the EC2 backend was initially accessed over HTTP. Direct requests therefore triggered browser mixed-content blocking.

13.5 Proxy Routing

A frontend/vercel.json configuration was implemented to route /api/* requests through the Vercel edge proxy to the EC2 backend. This allowed the browser to use the HTTPS frontend origin while Vercel handled backend routing.

13.6 Client Refactoring

DoctorDashboard, PatientDashboard, Login, and Register were refactored to replace hardcoded http://127.0.0.1:8000 URLs with relative /api endpoints.

13.7 CI/CD Result

The final frontend is connected to GitHub, allowing updates to the configured branch to trigger a new Vercel build and deployment.

14. TESTING, VALIDATION & END-TO-END DEMONSTRATION

14.1 Functional Verification

Test Area	Verification
Frontend	Public Vercel application loads.
Registration	Registration reaches cloud backend.
Authentication	Login workflow communicates with deployed API.
Symptom Submission	Symptoms are submitted through deployed path.
ML Prediction	Cloud backend executes bundled ML model.
Swagger	FastAPI documentation is accessible.
Network Routing	Relative /api requests route from Vercel to EC2.
Container Runtime	Dockerized backend runs on EC2.

14.2 Demonstration Flow

29. Open public MedAssist AI frontend.
30. Register or log in as patient.
31. Enter profile information.
32. Enter symptoms and submit analysis.
33. Observe potential disease prediction and confidence.
34. Review risk, recommendation, treatment/advisory and report.
35. Log in through healthcare-provider workflow.
36. Select a patient and review records/history.
37. Review analytics and health trends.
38. Use deployed Swagger interface to inspect API availability.

14.3 Performance Metrics

The project requirements specify model metrics including accuracy, precision, recall and F1-score, and system metrics including API response time, dashboard loading speed and concurrent-user handling. Exact numerical benchmark values should be reported only when measured from the actual model/system.

14.4 Validation Statement

The production communication path for registration, authentication, symptom submission and real-time ML disease prediction was successfully verified. This confirms that the major frontend-to-backend-to-model path operates outside the local environment.

15. RESULTS, LIMITATIONS, FUTURE SCOPE & CONCLUSION

15.1 Results

MedAssist AI was developed from a basic healthcare web application into a deployed AI-assisted platform. The completed system combines patient and healthcare-provider workflows, database-backed records, machine learning disease prediction, confidence and risk presentation, disease-specific advisory information, analytics, health trends, reporting, Docker containerization, AWS backend hosting, and Vercel frontend deployment.

15.2 Live Deliverables

Live Frontend Application: <https://med-assist-ai-delta.vercel.app/>
Interactive FastAPI Swagger Documentation: <http://3.110.188.253/docs>

15.3 Limitations

- The prediction model provides potential disease classifications and is not a substitute for clinical diagnosis.
- Prediction quality depends on training data, feature representation and model limitations.
- The deployed prototype uses SQLite rather than a managed production database.
- The cloud configuration is intended for academic demonstration, not certified healthcare production.
- Detailed concurrent-user and response-time benchmarks require dedicated load testing.
- Healthcare recommendations are informational and should be reviewed by qualified professionals.

15.4 Future Scope

- Migrate production database to PostgreSQL or another managed database.
- Introduce stronger JWT authentication and role authorization.
- Add HTTPS at the backend through a managed reverse proxy/load balancer.
- Add structured audit logging and monitoring.
- Use larger and clinically representative datasets for validation.
- Compare additional ML algorithms using controlled evaluation data.
- Improve prediction explainability.
- Add model monitoring and retraining workflows.
- Introduce stronger privacy and security controls for real healthcare environments.

15.5 Conclusion

The MedAssist AI project demonstrates integration of artificial intelligence with a complete web application and cloud deployment workflow. Across four milestones, the project progressed from core application development to AI prediction, healthcare recommendations and analytics, and finally Docker-based AWS backend deployment and Vercel frontend deployment. The final platform provides an end-to-end demonstration of symptom analysis, potential disease prediction, patient/doctor workflows, analytics, and cloud delivery.

References

- [FastAPI Documentation.](#)
- [React Documentation.](#)
- [Scikit-learn Documentation.](#)
- [Docker Documentation.](#)
- [AWS EC2 Documentation.](#)
- [Vercel Documentation.](#)
- [Python, SQLAlchemy, NumPy, Pandas and Joblib documentation.](#)